

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ
ПО ФИНАЛЬНОЙ РАБОТЕ
ТЕКСТОВАЯ RPG-ИГРА
по дисциплине
«Информатика и программирование»

Студент		
гр. БИС-25-3	_____	О.И. Шестопалова
Ассистент		
преподавателя	_____	М.В. Водяницкий

Владивосток 2026

Задание

Выполнить задания на Python и оформить отчет по стандартам ВВГУ.

Финальная лабораторная работа.

Финальная лабораторная работа состоит из выполнения задания по техническому заданию (ТЗ), оформления отчета по СТО и очной защиты выполненной работы (код и отчет)

Варианты работы:

- RPG текстовая игра
- Система управления заправочной станцией [(Тех. Задание)]

Вариант работы выбирается студентом самостоятельно

Техническое задание - Текстовая RPG-игра.

Вы работаете программистом в небольшой японской компании на заре игровой индустрии.

Компания разрабатывает свою первую экспериментальную игру - текстовую RPG, которая должна запускаться прямо в консоли и погружать игрока в атмосферу подземелий, опасностей и развития персонажа

Ваша задача - реализовать прототип игры, который демонстрирует основные игровые механики: характеристики персонажа, бои, прокачку, инвентарь и случайные события

- Создание персонажа (выбор расы, характеристики персонажа)
- Опыт и уровни (прокачка характеристик)
- Инвентарь и экипировка
- Подземелье и комнаты (структура, типы комнат, видимость комнат)
- Враги и сложность
- Боевая система
- Предметы и добыча
- Хранение данных

Критерии оценивания

1. Соответствие требованиям из ТЗ
2. Правильная и стабильная работа программы (отсутствие аварийных завершений, корректная обработка ошибок и некорректных входных данных)

3. Чистота и понятность кода (логичная структурированность, использование функций, наличие комментариев)

4. Оформление отчета по СТО

5. Содержание отчета с полным и понятным описанием работы программы (пояснены все основные сложные функции, описана общая структура работы программы)

6. Очная защита работы (ответы студента на вопросы по программе, отчету и исходному коду)

7. Наличие опубликованного исходного кода в учебном git репозитории (включая цифровую версию отчета, docx или pdf)

При наличии замечаний по любому из критериев итоговый балл за работу может быть снижен

Защита работы

Отчет должен быть сдан в печатном виде:

- преподавателю
- или на кафедру (1448)

Допускается двухсторонняя печать ЧБ или цветная

Если в отчете допущены ошибки, связанные с СТО, или имеются проблемы с содержанием - отчет не принимается и студент не допускается к защите финальной работы. Отчет можно переделать и попробовать сдать повторно

После сдачи отчета студент допускается к защите финальной работы. На защите будет проверена работоспособность программы, а также будут заданы вопросы студенту по коду программы, отчету и работе самой программы

Код программы и цифровая версия отчета (docx или pdf) должны быть опубликованы в git репозитории

Работа считается сданной при наличии подписи преподавателя и выставленного балла за раб

Содержание

Введение	3
1. Выполнение работы	4
1.1 Создание персонажа	5
1.2. Класс персонажа	7
1.3 Генерация подземелья и комнат	8
1.4 Боевая система	9
1.5 Комната с сундуком	10
1.6 Комната отдыха	11
1.7 Инвентарь и экипировка	11
1.8 Система опыта и повышения уровня	12
1.9 Основной цикл игры	13
2. Тестирование	15
2.1 Особый функционал	15
2.2 Интерфейс во время тестирования	16
Заключение	19

Введение

Современные программные системы всё чаще ориентированы на интерактивное взаимодействие с пользователем и моделирование сложных процессов в упрощённой форме. Одним из наглядных примеров таких систем являются компьютерные игры, в частности текстовые RPG-игры, которые позволяют реализовать широкий спектр алгоритмических и объектно-ориентированных решений в рамках консольных приложений.

Текстовые RPG-игры представляют собой удобную среду для изучения основ программирования, так как включают в себя работу с пользовательским вводом, условными операторами, циклами, структурами данных, функциями и классами. Кроме того, разработка подобной игры позволяет ознакомиться с принципами проектирования программных систем, такими как модульность, инкапсуляция и повторное использование кода.

Целью данной работы является разработка консольной текстовой RPG-игры, моделирующей процесс исследования подземелья, пошаговых боёв, развития персонажа и управления инвентарём. В рамках работы была поставлена задача реализовать программный прототип, демонстрирующий основные игровые механики и обеспечивающий устойчивую работу при различных сценариях взаимодействия пользователя с системой.

В процессе выполнения работы были реализованы механизмы создания персонажа с выбором расы и генерацией характеристик, пошаговая боевая система, система опыта и уровней, генерация случайных комнат подземелья, а также управление инвентарём и экипировкой персонажа.

Программа разработана на языке Python и предназначена для работы в консольном режиме. Выбор данного языка программирования обусловлен его простотой, наглядностью синтаксиса и широкими возможностями для разработки прототипов программных систем.

1. Выполнение работы

При разработке я старался придерживаться нескольких простых правил:

- Программа должна быть понятной — как для пользователя, так и для того, кто будет читать код.
- Нельзя допускать «падений» — даже если пользователь введёт что-то неправильно, программа спокойно обрабатывает ошибку и предлагает повторить ввод.
- Все данные должны сохраняться до выхода из игры.

Эти правила помогли сделать программу удобной и надёжной.

Данный проект был выполнен по теме RPG текстовая игра. В коде был реализована боевая система, выпадение предметов, торговля, характеристики персонажа, а также инвентарь.

Благодаря модулю «import random», который вводится в самом начале кода, была реализована генерация случайных характеристик, врагов, предметов и локаций. На рисунке 1 представлен пример реализации модуля.

```

174 class Enemy:
175     def __init__(self, floor):
176         types = ["Слизь", "Скелет", "Паук", "Гоблин", "Скелет", "Паук", "Гоблин", "Орк"]
177         self.name = random.choice(types)
178         if self.name == "Слизь":
179             self.level = random.randint(max(1, floor-1), floor+1)
180             self.hp = random.randint(10, 15) + floor**2
181             self.max_hp = self.hp
182             self.attack = random.randint(0, 1) + floor
183             self.defense = random.randint(0, 1) + floor
184             self.exp_reward = random.randint(5, 10) + floor**2
185             self.coin_reward = random.randint(1, 5) + floor
186         elif self.name == "Орк":
187             self.level = random.randint(max(1, floor-1), floor+1)
188             self.hp = random.randint(50, 70) + floor**2 * 2
189             self.max_hp = self.hp
190             self.attack = random.randint(15, 20) + floor * 8
191             self.defense = random.randint(5, 10) + floor
192             self.exp_reward = random.randint(40, 80) + floor * 8
193             self.coin_reward = random.randint(20, 30) + floor * 5
194         else:
195             self.level = random.randint(max(1, floor-1), floor+1)
196             self.hp = random.randint(20, 40) + floor**2 * 2
197             self.max_hp = self.hp
198             self.attack = random.randint(5, 10) + floor * 5
199             self.defense = random.randint(0, 5) + floor
200             self.exp_reward = random.randint(20, 40) + floor * 5
201             self.coin_reward = random.randint(5, 20) + floor * 3

```

Рисунок 1 — Листинг программы

В данном случае модуль «import random» используется для генерации случайных характеристик для разных типов врагов перед входом в боевую комнату.

На рисунке 2 ещё один пример реализации модуля «import random».

```

326         dodge_chance = player.agility/100 + 0.3
327         if random.random() < dodge_chance:
328             player_damage = max(1, player.attack + random.randint(-2, 3)) * 2
329             enemy.hp -= player_damage
330             print(f"Вы уклонились от атаки врага! Вы нанесли {player_damage} урона!")

```

Рисунок 2 — Листинг программы

Через генерацию случайного урона и шанса на уклонение во время боя.

В завершение примером использования модуля послужит рисунок 3.

```

405     if treasure_type in ["item", "both"]:
406         items = ["Малое зелье здоровья", "Среднее зелье здоровья", "Большое зелье здоровья", "Малое зелье силы",
407                 "Среднее зелье силы", "Большое зелье силы", "Малое зелье ловкости", "Среднее зелье ловкости",
408                 "Большое зелье ловкости", "Малое зелье защиты", "Среднее зелье защиты", "Большое зелье защиты",
409                 "Волшебный меч", "Магический доспех", "Эльфийский лук"]
410         item = random.choice(items)
411         player.inventory.append(item)
412         print(f"Найдено: {item}")

```

Рисунок 3 — Листинг программы

Реализации модуля представлена в виде генерация случайного предмета и списка для комнаты с сундуком.

1.1 Создание персонажа

Функция `create_character()` реализует начальный этап игры и отвечает за создание игрового персонажа пользователя. Реализация этой функции показана на рисунке 4.

```

209 def create_character():
210     character = Character()
211
212     print("СОЗДАНИЕ ПЕРСОНАЖА")
213     print("Выберите расу:")
214     print("1. Человек")
215     print("2. Эльф")
216     print("3. Дворф")
217     print("4. Узнать информацию о расах")
218
219     while character.race is None:
220         try:
221             choice = int(input("\nВаш выбор: "))
222             if choice == 1:
223                 character.race = "Человек"
224                 character.height = random.randint(165, 185)
225                 character.weight = random.randint(65, 85)
226                 character.max_hp = random.randint(80, 100)
227                 character.hp = character.max_hp
228                 character.attack = random.randint(8, 12)
229                 character.defense = random.randint(5, 8)
230                 character.agility = random.randint(10, 14)

```

Рисунок 4 — Листинг программы

В начале выполнения функции пользователю выводится меню выбора расы персонажа. Выбор осуществляется из трёх вариантов:

- человек
- эльф
- дворф

Выбор пользователя обрабатывается с проверкой корректности ввода, что исключает возможность возникновения ошибок выполнения программы.

После выбора расы выполняется генерация стартовых характеристик персонажа. Характеристики формируются случайным образом в пределах диапазонов, заданных для каждой расы, что обеспечивает разнообразие игровых сценариев. На рисунке 4 представлен диапазон для человека.

Генерируются следующие параметры:

- максимальное и текущее здоровье;
- сила атаки;
- защита;
- ловкость;
- рост;
- вес.

Дополнительно в функции реализована логика влияния физических параметров персонажа на его игровые характеристики. Рассчитывается индекс массы тела, который может увеличивать или уменьшать ловкость персонажа. Данный механизм добавляет элемент реализма и вариативности в игровой процесс.

Функция `show_stats()` помогает отображать актуальные характеристики по ходу всей игры. Реализация показана на рисунке 5.

```

21     def show_stats(self):
22         print(f"\nХАРАКТЕРИСТИКИ")
23         print(f"Раса: {self.race}")
24         print(f"Уровень: {self.level}, Опыт: {self.exp}/{self.exp_to_next}")
25         print(f"Очки характеристик: {self.skill_points}")
26         print(f"HP: {self.hp}/{self.max_hp}")
27         print(f"Атака: {self.attack}")
28         print(f"Защита: {self.defense}")
29         print(f"Ловкость: {self.agility}")
30         print(f"Рост: {self.height} см")
31         print(f"Вес: {self.weight} кг")
32         print(f"Монеты: {self.coins}")
33
34         if self.equipped["weapon"]:
35             print(f"Оружие: {self.equipped['weapon']}")
36         if self.equipped["armor"]:
37             print(f"Броня: {self.equipped['armor']}")

```

Рисунок 5 — Листинг программы

По завершении создания персонажа выводится подробная информация о всех сгенерированных характеристиках.

1.2. Класс персонажа

Класс Character предназначен для хранения и управления состоянием игрового персонажа. Реализация на рисунке 6.

```

3  class Character:
4      def __init__(self):
5          self.race = None
6          self.level = 1
7          self.exp = 0
8          self.exp_to_next = 100
9          self.skill_points = 0
10         self.hp = 0
11         self.max_hp = 0
12         self.attack = 0
13         self.defense = 0 # Защита
14         self.agility = 0 # Ловкость
15         self.height = 0 # Рост
16         self.weight = 0 # Вес
17         self.inventory = []
18         self.equipped = {"weapon": None, "armor": None}
19         self.coins = 0

```

Рисунок 6 — Листинг программы

В данном классе содержатся следующие группы данных:

- основные характеристики персонажа;
- информация об уровне и опыте;
- количество очков прокачки;
- инвентарь и экипировка;
- количество внутриигровых монет.

Данные переменные важны для отображения актуальных данных персонажа.

В классе реализованы методы:

- получения урона с учётом защиты;
- восстановления здоровья;
- получения опыта;
- повышения уровня;
- распределения очков прокачки;
- покупка предметов (для сокращения представлены только варианты выбора);
- отображения текущих характеристик (для сокращения представлены только варианты выбора).

Инкапсуляция логики персонажа в отдельный класс позволяет упростить управление состоянием игрока и повысить читаемость программного кода.

1.3 Генерация подземелья и комнат

Функция `generate_room()` отвечает за генерацию игровых комнат подземелья.

Реализация на рисунке 7.

```

276 # Генерация комнат
277 def generate_room(room_type=None):
278     if room_type is None:
279         room_type = random.choice(["combat", "rest", "treasure", "combat", "rest", "combat", "combat"])
280
281     rooms = {
282         "combat": "Боевая комната",
283         "rest": "Комната отдыха",
284         "treasure": "Комната с сундуком"
285     }
286
287     return {
288         "type": room_type, # Тип комнаты
289         "name": rooms[room_type],
290         "visited": False
291     }
292
293 def combat_room(player):
294     print("\nБОЕВАЯ КОМНАТА")
295     enemy = Enemy(player.level)
296     print(f"На вас напал {enemy.name}!")

```

Рисунок 7 — Листинг программы

Тип комнаты определяется случайным образом, что позволяет создать непредсказуемый игровой процесс. Для генерации используются взвешенные вероятности, в результате чего боевые комнаты встречаются чаще.

Каждая комната описывается структурой данных, содержащей:

- тип комнаты;
- название комнаты;
- признак посещения.

Перед выбором направления движения игроку может быть показана либо скрыта информация о типе следующей комнаты, что добавляет элемент случайности и принятия решений.

1.4 Боевая система

При входе в боевую комнату создаётся противник класса Enemy. Реализация кода на рисунке 8.

```

173 # Класс врага
174 class Enemy:
175     def __init__(self, floor):
176         types = ["Слизь", "Скелет", "Паук", "Гоблин", "Скелет", "Паук", "Гоблин", "Орк"]
177         self.name = random.choice(types)
178         if self.name == "Слизь":
179             self.level = random.randint(max(1, floor-1), floor+1)
180             self.hp = random.randint(10, 15) + floor**2
181             self.max_hp = self.hp
182             self.attack = random.randint(0, 1) + floor
183             self.defense = random.randint(0, 1) + floor
184             self.exp_reward = random.randint(5, 10) + floor**2
185             self.coin_reward = random.randint(1, 5) + floor

```

Рисунок 8 — Листинг программы

Характеристики врага зависят от текущего уровня персонажа, что обеспечивает постепенное увеличение сложности игры (на рисунке 10 представлены диапазоны характеристик для типа врага «Слизь»).

Боевая система реализована в функции `combat_room()` и является одной из ключевых частей игрового процесса. Реализация кода на рисунке 9.

```

293 def combat_room(player):
294     print("\nБОЕВАЯ КОМНАТА")
295     enemy = Enemy(player.level)
296     print(f"На вас напал {enemy.name}!")
297
298     while enemy.hp > 0 and player.hp > 0: # Если ещё оба живы
299         enemy.show_stats()
300         print(f"Ваше HP: {player.hp}/{player.max_hp}")
301         print("\nДоступные действия:")
302         print("1. Атаковать")
303         print("2. Попытаться уклониться")
304         print("3. Сбежать")

```

Рисунок 9 — Листинг программы

В процессе боя игроку доступны следующие действия:

- стандартная атака;
- попытка уклонения от атаки противника;
- побег с поля боя.

Для уклонения существует определённая вероятность успеха, которая для случайного уклонения равна ловкости, делёной на сто, а при намеренном уклонении шанс повышается на 30%. Побег при любых условиях равен 30%.

При расчёте урона учитываются:

- значение атаки атакующего;

- значение защиты защищающегося;
- случайные модификаторы;
- вероятность уклонения, зависящая от ловкости.

После победы над врагом игрок получает опыт, монеты, а также имеет вероятность получить случайный предмет. В случае поражения игровой процесс завершается.

1.5 Комната с сундуком

Функция `treasure_room()` реализует механику получения наград. Реализована на рисунке 10.

```

394 def treasure_room(player):
395     print("\nКОМНАТА С СУНДУКОМ")
396     print("Вы нашли сундук!")
397
398     treasure_type = random.choice(["coins", "item", "both"])
399
400     if treasure_type in ["coins", "both"]:
401         coins = random.randint(20, 50) + player.level * 10
402         player.coins += coins
403         print(f"Найдено монет: {coins}")
404
405     if treasure_type in ["item", "both"]:
406         items = ["Малое зелье здоровья", "Среднее зелье здоровья", "Большое зелье здоровья", "Малое зелье силы",
407                 "Среднее зелье силы", "Большое зелье силы", "Малое зелье ловкости", "Среднее зелье ловкости",
408                 "Большое зелье ловкости", "Малое зелье защиты", "Среднее зелье защиты", "Большое зелье защиты",
409                 "Волшебный меч", "Магический доспех", "Эльфийский лук"]
410         item = random.choice(items)
411         player.inventory.append(item)
412         print(f"Найдено: {item}")
413
414     print(f"Ваши монеты: {player.coins}")
415
416     use = input("Хотите открыть инвентарь? (да/нет): ").lower()
417     if use == "да":
418         manage_inventory(player)

```

Рисунок 10 — Листинг программы

В комнате с сундуком случайным образом определяется тип награды:

- только монеты;
- только предмет;
- комбинация монет и предметов.

Размер награды масштабируется в зависимости от уровня персонажа, что поддерживает баланс игры. Так же в данной комнате можно использовать или экипировать предметы.

1.6 Комната отдыха

Функция `rest_room()` реализует безопасные зоны подземелья. Реализована на рисунке 11.

```

421 def rest_room(player):
422     print("\nКОМНАТА ОТДЫХА")
423     print("Здесь безопасно. Вы можете отдохнуть.")
424
425     rest_amount = player.max_hp * 0.3
426     player.heal(int(rest_amount))
427     print(f"Вы восстановили {int(rest_amount)} HP")
428
429     if player.coins > 100 and random.random() < 0.5:
430         print(f"Вы встретили торговца! Монет: {player.coins}")
431         use = input("Хотите подойти? (да/нет): ").lower()
432         if use == "да":
433             player.use_coins()
434
435     if player.skill_points > 0:
436         use = input("Хотите использовать очки прокачки? (да/нет): ").lower()
437         if use == "да":
438             player.use_skill_points()
439
440     use = input("Хотите открыть инвентарь? (да/нет): ").lower()
441     if use == "да":
442         manage_inventory(player)

```

Рисунок 11 — Листинг программы

В данных комнатах персонаж восстанавливает часть здоровья, может распределить накопленные очки прокачки, улучшая выбранные характеристики, с 50% шансом можно приобрести предметы у торговца, а также воспользоваться инвентарём.

Комнаты отдыха играют важную роль в управлении ресурсами персонажа и подготовке к последующим боям.

1.7 Инвентарь и экипировка

Функция `manage_inventory()` реализует систему управления инвентарём. Реализована на рисунке 12.

```

444 def manage_inventory(player):
445     while True:
446         print("\nИНВЕНТАРЬ")
447         print(f"Монеты: {player.coins}")
448         print("Предметы:")
449
450         if not player.inventory:
451             print("Инвентарь пуст")
452         else:
453             for i, item in enumerate(player.inventory, 1):
454                 print(f"{i}. {item}")
455
456         print("\nЭкипировано:")
457         print(f"Оружие: {player.equipped['weapon'] or 'Нет'}")
458         print(f"Броня: {player.equipped['armor'] or 'Нет'}")
459
460         print("\n1. Использовать предмет")
461         print("2. Выбросить предмет")
462         print("3. Экипировать предмет")
463         print("4. Снять экипированный предмет")
464         print("5. Показать характеристики")
465         print("6. Выйти")

```

Рисунок 12 — Листинг программы

Игрок может:

- просматривать список предметов;
- использовать зелья;
- выбрасывать предметы;
- экипировать оружие и броню;
- снять оружие и броню;
- просмотреть свои характеристики.

Так же можно покинуть игру в любой момент и вывести окно с окончанием игры.

Инвентарь можно запустить по желанию игрока из комнаты с сундуком или комнаты отдыха.

1.8 Система опыта и повышения уровня

Система развития персонажа реализована функциями `add_exp()` и `level_up()`.

Реализация кода на рисунке 13.

```

49     # Получение опыта
50     def add_exp(self, amount):
51         self.exp += amount
52         print(f"Получено опыта: {amount}")
53         while self.exp >= self.exp_to_next:
54             self.level_up()
55
56     # Получение нового уровня
57     def level_up(self):
58         self.level += 1
59         self.exp -= self.exp_to_next
60         self.exp_to_next = self.exp_to_next * 1.5
61         self.skill_points += 1
62         self.max_hp += 10
63         self.hp = self.max_hp
64         print(f"Уровень ПОВЫШЕН до {self.level}!")
65         print(f"Очков характеристик: {self.skill_points}")

```

Рисунок 13 — Листинг программы

При повышении уровня:

- увеличивается уровень персонажа;
- возрастает максимальное здоровье;
- выдаются очки прокачки;
- полностью восстанавливается здоровье.

Уровень напрямую влияет на характеристики и стоимость предметов у торговца.

1.9 Основной цикл игры

Запуск игры происходит благодаря следующей части кода. Реализация на рисунке 14.

```

670     if __name__ == "__main__":
671         try:
672             main_game()
673         except KeyboardInterrupt:
674             print("\n\nИгра прервана")
675         except Exception as e:
676             print(f"Произошла ошибка: {e}")

```

Рисунок 14 — Листинг программы

Данный фрагмент всегда описывается в конце имеющегося кода и запускает функцию `main_game()`. В свою очередь функция `main_game()` управляет всей логикой игры.

Ниже представлена функция `main_game()`. Реализована на рисунке 15.

```

606 def main_game():
607     print("Загрузка игры...")
608     print("RPG. Добро пожаловать в подземелье!\n")
609
610     player = create_character()
611     current_floor = 1
612     rooms_cleared = 0
613
614     # Начальные предметы
615     player.inventory.append("Малое зелье здоровья")
616     player.inventory.append("Малое зелье силы")
617
618     while player.hp > 0:
619         print(f"\n=== ЭТАЖ {current_floor} ===")
620         print(f"Комнат пройдено: {rooms_cleared}")
621         player.show_stats()
622
623         # Генерация комнат на развилке
624         left_room = generate_room()
625         right_room = generate_room()
626
627         # Видимость комнат
628         left_visible = random.choice([True, False])
629         right_visible = random.choice([True, False])
630
631         print("\nПеред вами развилка:")
632         print(f"1. Слева: {left_room['name']} if left_visible else '???')")
633         print(f"2. Справа: {right_room['name']} if right_visible else '???')")
634         print("3. Выйти из игры")

```

Рисунок 15 — Листинг программы

В данной функции описаны: начало игры, начальные значения этажа и комнат, начальные предметы, генерация комнат подземелья и их видимость.

На рисунке 16 представлены обработка выбора комнат и механика перехода между этажами.

```

652     # Обработка выбранной комнаты
653     if selected_room["type"] == "combat":
654         if not combat_room(player): # Если проиграли
655             print("Игра окончена!")
656             break
657     elif selected_room["type"] == "treasure":
658         treasure_room(player)
659     elif selected_room["type"] == "rest":
660         rest_room(player)
661
662     rooms_cleared += 1
663
664     # Переход на следующий этаж
665     if rooms_cleared % 5 == 0:
666         current_floor += 1
667         print(f"\nВы спустились на ЭТАЖ {current_floor}!")
668         print("Враги стали сильнее!")
669         player.heal(player.max_hp // 2) # На 50% HP
670         print(f"Вы восстановили {player.max_hp // 2} HP")
671
672     ...

```

Рисунок 16 — Листинг программы

Кракая и понятная обработка комнат даёт возможность в приятном для взгляда образе реализовать программное содержание файла `main_game()`.

2. Тестирование

Тестирование программы проводилось в интерактивном режиме путём последовательного прохождения игровых сценариев и анализа реакции системы на действия пользователя.

Основной целью тестирования являлась проверка корректности работы всех реализованных функций и устойчивости программы к некорректному пользовательскому вводу.

В ходе тестирования были проверены следующие сценарии:

- запуск программы и корректность отображения стартового меню;
- создание персонажей всех доступных рас;
- генерация характеристик в допустимых диапазонах;
- влияние роста и веса на параметр ловкости;
- корректность пошаговой боевой системы;
- использование предметов в бою;
- обработка победы и поражения;
- начисление опыта и повышение уровня;
- распределение очков прокачки;
- генерация комнат различных типов;
- работа комнат отдыха и восстановление здоровья;
- получение наград в комнатах с сундуками;
- управление инвентарём и экипировкой;
- переход между этажами подземелья;
- корректное завершение игры.

Отдельное внимание уделялось обработке некорректного ввода пользователя. При вводе недопустимых значений программа корректно выводит сообщения об ошибках и не допускает аварийного завершения.

По результатам тестирования установлено, что программа корректно реализует заявленный функционал, устойчиво работает при различных сценариях использования и соответствует требованиям технического задания.

2.1 Особый функционал

В данном проекте реализовано несколько индивидуальных особенностей. Ключевой индивидуальной особенностью является система загадочного торговца. Этот персонаж появляется нерегулярно, подчиняясь особым условиям. Его появление — это всегда значимое событие, дающее игроку доступ к эксклюзивным возможностям. Во-

первых, у него можно приобрести безвозвратно утраченные знания (дополнительные очки характеристик), что позволяет целенаправленно усиливать героя вне стандартного прогресса. Во-вторых, он предлагает эликсиры просветления, дарующие существенно увеличенный опыт за победы, что ускоряет развитие. Но самое интересное — это его ассортимент зелий.

Система врагов выстроена по чёткой иерархии сложности, что делает каждый бой осмысленным. Враги делятся на три выраженные категории:

- Орки – самые мощные и опасные враги, обладающие высоким здоровьем, уроном и хорошими выпадающими предметами при победе.
- Гоблины, Скелеты, Пауки – усреднённый тип, составляющий основу механики боя. Их характеристики сбалансированы и близки друг к другу.
- Слизни – самые слабые враги, служащие источником низкосортных ресурсов и опыта.

Во время боевой системы наглядно показывается генерация случайного уклонения, повышенного урона и попытки побега.

Важнейшей индивидуальной механикой является динамическая система прогрессии сложности. Сила всех врагов в игре не фиксирована — она напрямую и не линейно зависит от глубины этажа подземелья, на котором находится игрок.

2.2 Интерфейс во время тестирования

Игра выполнена в минимализме. Основное окно при загрузке показано на рисунке 17.

```
Загрузка игры...
RPG. Добро пожаловать в подземелье!

СОЗДАНИЕ ПЕРСОНАЖА
Выберите расу:
1. Человек
2. Эльф
3. Дворф
4. Узнать информацию о расах

Ваш выбор: █
```

Рисунок 17 — Листинг программы

У пользователя также всегда есть возможность узнать более подробную информацию о классах персонажей.

После создания персонажа выводится окно с характеристиками. Показа на рисунке 18.

```

Ваш выбор: 1
Персонаж создан!

ХАРАКТЕРИСТИКИ
Раса: Человек
Уровень: 1, Опыт: 0/100)
Очки характеристик: 0
НР: 87/87
Атака: 9
Защита: 6
Ловкость: 12
Рост: 174 см
Вес: 77 кг
Монеты: 0

```

Рисунок 18 — Листинг программы

Начало положено. Для пользователя выводится окно с получившимися характеристиками.

Далее пользователю дают свободу действий. Представлено на рисунке 19.

```

Перед вами развилка:
1. Слева: Комната отдыха
2. Справа: ???
3. Выйти из игры
Куда пойти? █

```

Рисунок 19 — Листинг программы

Случайным образом выбирается комната слева (боевая комната, комната с сундуком, комната отдыха). Так же генерируется видимость комнат. В данном случае комната видна. Аналогичным образом выбирается комната справа без видимости.

Теперь разберём подробней каждый пункт.

Начнём с боевой комнаты, представленной на рисунке 20.

```

Вы идете налево...

БОЕВАЯ КОМНАТА
На вас напал Орк!

Ваш противник: Орк (Уровень 1)
НР: 56/56
Атака: 23, Защита: 8
Ваше НР: 87/87

Доступные действия:
1. Атаковать
2. Попытаться уклониться
3. Сбежать
Выберите действие: █

```

Рисунок 20 — Листинг программы

В комнате может попасться несколько типов монстров. Слабый – слизь, сильный – орк и обычные монстры – скелет, гоблин или паук. У каждого врага соответствующая сложность и награда. После создания врага у пользователя появляется выбор действий (атаковать, попытаться уклониться или сбежать).

Так же затронем комнату с сундуком, представленной на рисунке 21.

Вы идете направо...

КОМНАТА С СУНДУКОМ

Вы нашли сундук!

Найдено монет: 49

Найдено: Эльфийский лук

Ваши монеты: 144

Хотите открыть инвентарь? (да/нет): ☐

Рисунок 21 — Листинг программы

В комнате с сундуком вы со 100% вероятностью встречаете сундук. В нем могут попасться как предметы (зелья, оружие и броня), так и монеты. Имеется возможность воспользоваться инвентарём.

И наконец затронем последнюю комнату – комнату отдыха. Представлена на рисунке 22.

Вы идете налево...

КОМНАТА ОТДЫХА

Здесь безопасно. Вы можете отдохнуть.

Вы восстановили 27 HP

Вы встретили торговца! Монет: 196

Хотите подойти? (да/нет): нет

Хотите использовать очки прокачки? (да/нет): нет

Хотите открыть инвентарь? (да/нет): ☐

Рисунок 22 — Листинг программы

При входе в комнату отдыха восстанавливается 30% HP; При определённых условиях с шансом в 50% можно встретить торговца. Так же можно распределить очки прокачки или открыть инвентарь.

Заключение

В ходе выполнения данной работы был разработан программный проект, представляющий собой консольную текстовую RPG-игру, моделирующую процесс прохождения подземелья, включающий элементы боевой системы, развития персонажа и управления ресурсами.

В результате выполнения поставленных задач были реализованы основные игровые механики, такие как создание персонажа с выбором расы, генерация характеристик, пошаговые бои с противниками, система опыта и повышения уровня, работа с инвентарём и экипировкой, а также генерация случайных игровых событий.

Разработанная программа обладает логичной структурой, основанной на использовании классов и функций, что упрощает понимание кода и его дальнейшее сопровождение. Особое внимание было уделено обработке пользовательского ввода и предотвращению некорректных действий, что повышает устойчивость программы в процессе эксплуатации.

Проведённое тестирование показало, что программа корректно функционирует при различных сценариях игры, устойчиво обрабатывает ошибки ввода и соответствует требованиям технического задания.

Разработанный проект может быть использован в качестве учебного примера для изучения основ программирования, объектно-ориентированного подхода и разработки консольных приложений. В дальнейшем программа может быть расширена за счёт добавления системы сохранений, новых типов врагов, предметов и игровых механик.